Smallest Range Covering Elements From K Lists

4	10	15	24	26
0	9	12	20	
5	18	22	30	

or

4	10	15	24	26	0	9	12	20	5	18	22	30

[4,10] [4,15] ....

$(nk)^2 * (nk) \approx (nk)^3$

diff range selection combinations

minEle : 0

maxEle : 30

```
minEle        maxEle-1
i=0      i<=30-1
j=i+1    j<=30 (maxEle) {
```

[i,j] for this range check

bool ListEleVerification

iterating over all the K lists & checking if it contains single element that falls within this range [i,j].

if it returns 0, increment j & check for another range.

else if it returns 1, store the result [i,j].

} break j loop, continue i^{th} loop.

[0,5] Diff = 5

[1,9] Diff: 8

[4,9] Diff = 5

[20,24] Diff: 4

Brute-force Approach:

1. Range Selection.
2. Elements verification within the lists.

[i,j] — 1st range

(First ele) (last ele) } of each list

2nd range

Check if 2nd range falls within 1st range,

if not, break the loop j & increment i.

else // we know range 2 falls within range 1

& Obviously there gonna be elements within range 2, continue checking for other lists too,

& if for all lists, range 2 falls under range 1

Store i & j &

continue checking for other windows.

But this is specifically for scenarios where range 2 completely falls within range 1

Ex 1: [0,30] range 1

[4,26] range 2

[4,26] falls within [0,30]

But what if

Ex 2: range 1 : [0,5]

& range 2 : [4,26]

Obviously ele 4 falls within [0,5] so we can return 1 yes?

Ex 3: range 1 : [2,5]

range 2 : [1,20]

1	6	18	20

Range 2 may or may not contain element that falls within range 1. not sure X.

* Let's break it down into subproblem

⇒ Smallest Range Covering Elements from 2 lists

Sub problem: Smallest Range covering Elements from 2 lists.

[0,20] 2nd Range
[5,30]

0	9	12	20
5	18	22	30

minEle: 0
maxEle: 30

Brute-force:
i) Range selection
ii) Elements verification

i = minEle i <= maxEle - 1
j = i+1 j <= maxEle
1st Range [i,j] → List Verification

Ex:
i=0 i<=29
j=1 j<=30
i=0 j=1
[0,1] range1
[0,20] range2
range 2 contains Ele which falls within range 1.

But
Suppose
Range 1: [2,5]
Range 2: [1,20]

1	6	18	20

range2 may or may not contains Elements that falls within range1
What if it doesn't

in such scenarios, we have to manually iterate over the list & check for elements to confirm.

& Most often our ans will fall under this category

How? let's say

4	10	15	24	26
0	9	12	20	
5	18	22	30	

Ans: [20,24] range1

range2
[4,26] ✗ we are not sure by just looking at range.
[0,20] ✓
[5,30] ✗ same, not sure.

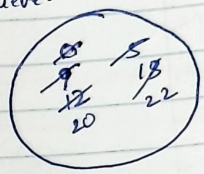
(MinHeap)

Approach2: We know Elements are sorted

0	9	12	20
5	18	22	30

[0,5] Diff: 5
Smallest of smaller list
Smallest of larger list.
→ [9,5] Diff: 4 → [9,18]
[12,18]
[12,22]
[20,22] Diff: 2 Ans

How to achieve:



Biggest Ele: 18

Diff	Range [top element, Biggest Ele]	Ans
5	[0,5]	[0,5]
4	[5,9]	[5,9]
9	[9,18]	[5,9]
6	[12,18]	[5,9]
2	[18,20]	[18,20]
2	[20,22] → ✗ 20 → ✗ Stop & return	[18,20]

Let's do for K lists.

Not Efficient, Neither Reliable Approach.
As we may have to manually iterate over lists & check.

Smallest Range covering Elements from K lists.

Approach 2: (using MinHeap).

2 conditions:

we define the range $[a,b]$ is smaller than range $[c,d]$ if

i) $b-a < d-c$

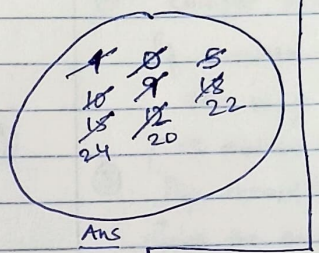
ii) $a < c$ if $b-a == d-c$

4	10	15	24	26
0	9	12	20	
5	18	22	30	

i) All lists, first Elements inserted

Biggest Ele: 5

ii) Comparing (pop the minimum & push it's adjoining Ele To minimize the diff & check for all possible diff ranges.



Ans

Diff	Ranges: [top Ele, Biggest Ele]	
5	[0,5]	[0,5]
5	[4,9]	[0,5]
5	[5,10]	[0,5]
9	[9,18]	[0,5]
8	[10,18]	[0,5]
6	[12,18]	[0,5]
5	[15,20]	[0,5]
6	[18,24]	[0,5]
4	[20,24]	[20,24]

Diff = 5 yet we go for [0,5] over [4,9] & [5,10]

20 → x so stop & return

Q: Now why we are using MinHeap? & What is the intuition behind it.

The Ques says: Smallest Range covering elements from K lists.

The Aim is to minimize the difference b/w Smallest Ele & Biggest Ele.

Initially we have range [0,5]

if we increment 5 (Biggest Ele) → Range increases which is not what we

& so because of this, we pop smallest Ele & insert it's adjoining element within that list, (to minimize the diff).

As we know each list is sorted in itself.

```
vector<int> smallestRange (vector<vector<int>>& nums) {

    // min heap
    //   value, row, col
    // pair<int, pair<int,int>>

priority_queue <pair<int, pair<int,int>>,
                vector<pair<int, pair<int,int>>>,
                greater<pair<int, pair<int,int>>>> p;

int minimum, maximum = INT_MIN;
// insert first element of each row into heap.
for (int i=0; i<nums.size(); i++)
{
  p.push (make_pair (nums[i][0], make_pair (i, 0)));
  maximum = max (maximum, nums[i][0]);
}

minimum = p.top().first;
vector<int> ans (2);
ans[0] = minimum;
ans[1] = maximum;

pair<int, pair<int,int>> temp;
int row, col, elem;
while ( p.size() == nums.size()) {   // until priority_q contains 1 element from each row/lists.

    temp = p.top();
    p.pop();
    elem = temp.first;
    row = temp.second.first;
    col = temp.second.second;

    if ( col+1 < nums[row].size()) {
        col++;
        p.push (make_pair (nums[row][col], make_pair (row, col)));
        maximum = max (maximum, nums[row][col]);
        minimum = p.top().first;
        // if I got smallest range, update.
        if (maximum - minimum < ans[1] - ans[0])
        { ans[0] = minimum;
          ans[1] = maximum;
        }
    } // while loop ends

return ans;

} // func ends
```

Psuedo Steps:

1. Push first element of each list into min-heap, track the currentMax among them.
2. Compute range using heap top (currentMin) & currentMax, update best ans if this range is smaller. (To minimize the diff/range),
3. Pop currentMin, push its next element from the same list (if exists), & update currentMax.
4. Repeat steps above until some list runs out of elements after a pop, return the best range (ans).